CASO PRÁCTICO: Sistema de Gestión de Alquiler de Vehículos

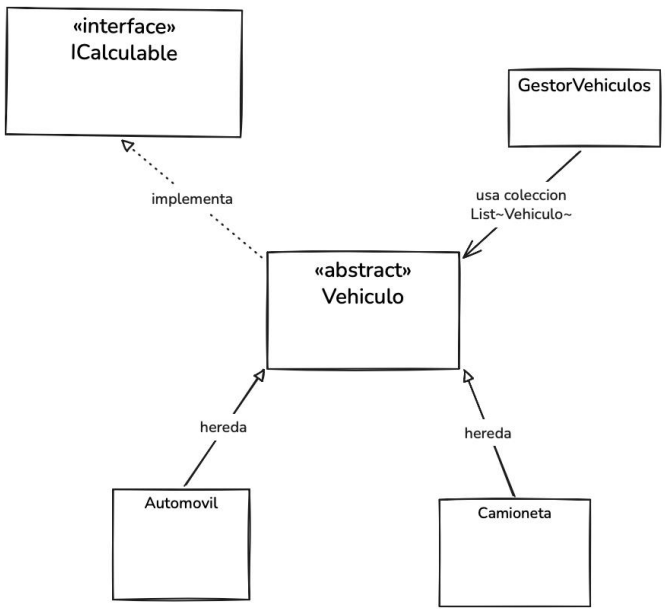
1. Contexto

Una empresa de *Rent-a-Car* requiere gestionar la flota de vehículos que tiene en arriendo. Dentro de la gestión se considera el cálculo de la tarifa final de alquiler y la administración de los vehículos disponibles.

Existen DOS tipos de vehículos: Automóvil y Camioneta.

El sistema se debe diseñar bajo un modelo orientado a objetos que aplique herencia, interfaces, colecciones y manejo de excepciones.

2. Diagrama de Clases (Estructura)



DSY Concepción

3. Requerimientos de Dominio y Métodos Especializados

Todas las clases deben incluir constructor(es), métodos accesadores (*getters*), mutadores (*setters*) y el método toString(). Los constructores deben delegar la asignación de valores a los métodos *setter* para asegurar las validaciones.

Todos los vehículos comparten:

- Patente (String)
- Días de arriendo
- Precio base diario

Los automóviles tienen un indicador para saber si tienen o no seguro premium. En el caso de las camionetas tienen una capacidad de carga expresada en toneladas.

Reglas de cálculo del arriendo (ICalculable):

1. Automóvil:
    a. Costo Base: dias * precio_*base*_diario
    b. Regla: Si tiene Seguro Premium entonces se le adiciona un cargo fijo de $15.000 al costo base; de lo contrario, se mantiene el costo base.

2. Camioneta:
    a. Regla de cálculo: días * precioBaseDiario + capacidadCarga x 3500

4. Validaciones Esperadas

Las validaciones deben aplicarse dentro de los métodos *setter* de cada clase. Si no se cumple una regla, debe lanzarse una excepción de tipo IllegalArgumentException con un mensaje descriptivo:

- Patente: No puede ser nula ni tener menos de 6 caracteres.
- Días de arriendo: Debe ser un número mayor a cero.
- Precio base diario: Debe ser un valor mayor a $15.000
- Capacidad de carga: Debe ser un valor positivo

DSY1102@Concepción

5. Operaciones del Gestor (GestorVehiculos)

La clase encargada de administrar la colección debe exponer las siguientes operaciones:

1. registrarVehiculo(Vehiculo v): Agrega un vehículo a la colección y quien llame al método debe informar por consola: "[Matrícula] ha sido registrado con éxito". Las patentes de los vehículos DEBEN SER únicas.
2. existeVehiculo(String patente): Retorna un valor booleano (true/false) para verificar si existe un vehículo dada su patente.

6. Requerimientos de la Clase Principal (Main)

1. Instanciar 4 vehículos utilizando los datos de la siguiente tabla:

Tipo	Matrícula	Días	Precio Diario	Atributo Específico
Automóvil	BRPY46	3	$25.000	Tiene seguro premium (true)
Camioneta	BBJS12	5	$40.000	Capacidad carga = 2.5 Toneladas
Camioneta	GHDD89	2	$35.000	Capacidad carga = 1.0 Toneladas
Automóvil	DDJD75	4	$22.000	No tiene seguro premium

2. Modificación previa al registro: Modificar el precio base diario de la camioneta BBJS12 aumentándolo en un 20% usando su método *setter*.
3. Registrar todos los vehículos en el GestorVehiculos.
4. Invocar las operaciones del gestor: buscar la patente GHDD89 e imprimir el listado general.

7. Salida Esperada por Consola

Gestión de Alquiler de Vehículos
** Agregando vehículos **
BRPY46 ha sido registrado con éxito
BBJS12 ha sido registrado con éxito
GHDD89 ha sido registrado con éxito

DSY1102 @concepción

DDJD75 ha sido registrado con éxito

*** LISTADO DE VEHÍCULOS ***
Matrícula BRPY46 - Días: 3 - Precio/Día: 25000.0 - Costo Arriendo: 90000.0
Matrícula BBJS12 - Días: 5 - Precio/Día: 48000.0 - Costo Arriendo: 248750.0
Matrícula GHDD89 - Días: 2 - Precio/Día: 35000.0 - Costo Arriendo: 73500.0
Matrícula DDJD75 - Días: 4 - Precio/Día: 22000.0 - Costo Arriendo: 88000.0

** Búsqueda por patente... **
El vehículo existe

DSY1102 @Concepción